

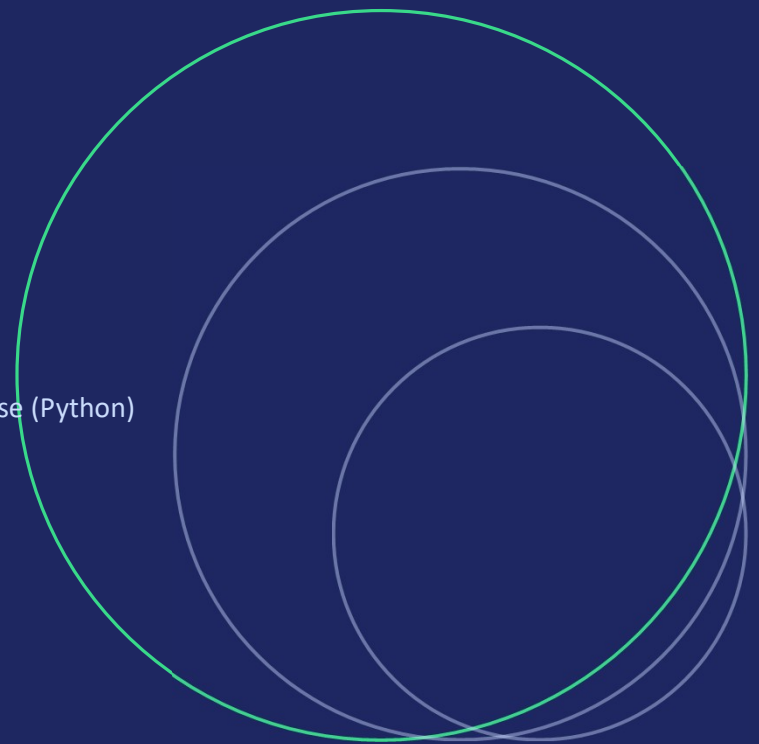
Writing Clean, Testable, Modular Code

Code that's easy to read, easy to change, and easy to trust

A hands-on training session with worked before/after examples, checklists, and a graded exercise (Python)

Fidelity, Gurgaon | Fresher Onboarding Session

Presented by Abinash Kumar Mishra · Co-Founder & CTO, AceRanker AI



What We'll Cover

01

Why Clean Code Matters

The real cost of code nobody can read or change safely

02

Clean Code Principles

Naming, small functions, single responsibility, avoiding duplication

03

Modularity

Separation of concerns, cohesion vs. coupling, dependency injection

04

Testability

What makes code hard to test — and how to design it to be easy

05

Exercise

Refactor a real (messy) module and add tests — graded against a rubric

Why Clean Code Matters

You write code once. You — or a teammate — read it dozens of times, and change it for years.

10x

more time is spent reading code than writing it

~60%

of engineering time goes to maintaining existing code, not writing new features

1 bug

in untested code can take down a production system serving millions

"Clean code always looks like it was written by someone who cares."

Robert C. Martin ("Uncle Bob"), Clean Code — Clean, testable, modular code isn't a luxury for later. It's what lets a team ship confidently, every sprint, for years.

Part 1

Clean Code Principles

The habits that make code readable at a glance

PRINCIPLE 01

Meaningful Names

Names should reveal intent — a reader shouldn't need a comment to know what a variable holds

AVOID

```
d = 4 # days

def calc(l, w):
    return l * w

if s == 1:
    process(x)
```

PREFER

```
days_since_signup = 4

def rectangle_area(length, width):
    return length * width

if status == Status.ACTIVE:
    process(order)
```

Good names eliminate whole categories of comments. If you need a comment to explain a name, rename it instead.

PRINCIPLE 02

Functions Do One Thing

A function should have a single, clear responsibility — not five loosely related ones

AVOID

```
def process_order(order):  
    validate(order)  
    total = 0  
    for item in order.items:  
        total += item.price  
    send_email(order.user,  
              'Order placed')  
    save_to_db(order, total)
```

PREFER

```
def process_order(order):  
    validate(order)  
    total = calculate_total(order)  
    save_order(order, total)  
    notify_customer(order)  
  
# each helper has ONE job,  
# is independently testable
```

If you can't describe a function in one short sentence without 'and', it's doing too much — split it.

PRINCIPLE 03

Avoid Duplication (DRY)

Repeated logic means every bug fix and change has to happen in multiple places

AVOID

```
def send_welcome_email(user):  
    conn = smtp.connect(HOST)  
    conn.send(user.email, WELCOME)  
  
def send_reset_email(user):  
    conn = smtp.connect(HOST)  
    conn.send(user.email, RESET)
```

PREFER

```
def send_email(user, template):  
    conn = smtp.connect(HOST)  
    conn.send(user.email, template)  
  
def send_welcome_email(user):  
    send_email(user, WELCOME)  
  
def send_reset_email(user):  
    send_email(user, RESET)
```

"Duplication is the primary enemy of a well-designed system" — every duplicate is a place a future fix can be forgotten.

PRINCIPLE 04

Guard Clauses over Nesting

Deeply nested conditionals hide the actual logic — flatten with early returns

AVOID

```
def get_discount(user):  
    if user is not None:  
        if user.is_active:  
            if user.orders > 5:  
                return 0.1  
            else:  
                return 0  
        else:  
            return 0  
    else:  
        return 0
```

PREFER

```
def get_discount(user):  
    if user is None:  
        return 0  
    if not user.is_active:  
        return 0  
    if user.orders <= 5:  
        return 0  
    return 0.1
```

Guard clauses handle edge cases up front, so the 'happy path' logic reads top-to-bottom without a pyramid of indentation.

Clean Code, At a Glance

Five habits that compound into dramatically more maintainable code



Name things by intent

user_count, not n or tmp



Don't repeat yourself

Extract shared logic into one place



Keep function signatures small

3 or fewer params; group related ones into an object



One function, one job

Extract until each function fits on screen



Flatten with guard clauses

Handle edge cases first, keep the happy path linear

Part 2

Modularity

Designing systems out of independent, well-defined pieces

What Makes Code "Modular"?

A module is a unit of code with a clear boundary: what it does, and what it exposes to the rest of the system

1

High cohesion

Everything inside a module is closely related to one purpose

2

Low coupling

Modules depend on each other through small, stable interfaces — not internals

3

Clear boundary

A well-named public API; implementation details stay private

4

Independently testable

You can exercise a module's logic without spinning up the whole system

High Cohesion, Low Coupling

The goal shape of a well-modularized system



Each module hides its own details and talks to the others through a small, stable interface. Change how Payments calculates tax internally, and Orders never notices — as long as the interface doesn't change.

Dependency Injection

Pass in what a class needs instead of creating it internally — this is what makes modules swappable and testable

AVOID

```
class OrderService:
    def __init__(self):
        self.db = PostgresDB()
        self.mailer = Smtplib()

    def place(self, order):
        self.db.save(order)
        self.mailer.send(...)
```

PREFER

```
class OrderService:
    def __init__(self, db, mailer):
        self.db = db
        self.mailer = mailer

    def place(self, order):
        self.db.save(order)
        self.mailer.send(...)
```

Now OrderService doesn't know or care whether db is Postgres or an in-memory fake — swap it freely in tests or in production.

Separation of Concerns: Layers

Each layer has one job and only talks to the layer directly below it

Presentation / API

Handles HTTP requests, input parsing, response formatting

Business Logic

Core rules: pricing, validation, workflows — no I/O here

Data Access

Talks to the database, cache, or external services

Infrastructure

Database, message queue, third-party APIs

Part 3

Testability

Designing code so tests are easy to write and trust

What Makes Code Hard to Test?



Hidden dependencies

A function silently reaches out to a database, clock, or network call



Global / mutable state

Tests interfere with each other because they share state



Giant functions

One function doing ten things means one test needs ten setups



Side effects mixed with logic

Can't check the calculation without also triggering an email or a write

Prefer Pure Functions

A pure function's output depends only on its inputs, with no side effects — trivially testable

```
cart_total = 0

def add_to_cart(item):
    global cart_total
    cart_total += item.price
    log_to_db(item)
    return cart_total
```

AVOID

```
def calculate_total(items):
    return sum(i.price
               for i in items)

# pure: same input ->
# always same output,
# no side effects to fake
```

PREFER

Push side effects (DB writes, logging, I/O) to the edges of your system; keep the core logic pure and easy to assert on.

A Basic Unit Test (pytest)

Arrange → Act → Assert: set up inputs, call the function, check the result



pytest

```
# calculator.py
def calculate_total(items):
    return sum(item.price for item in items)

# test_calculator.py
def test_calculate_total_sums_item_prices():
    # Arrange
    items = [Item(price=10), Item(price=25)]

    # Act
    result = calculate_total(items)

    # Assert
    assert result == 35
```

Testing with Fakes & Mocks

Because OrderService takes its db as a parameter, tests can swap in a fake — no real database needed

```
class FakeDB:
    def __init__(self):
        self.saved = []
    def save(self, order):
        self.saved.append(order)

def test_place_order_saves_to_db():
    fake_db = FakeDB()
    service = OrderService(fake_db, FakeMailer())

    service.place(Order(id=1))

    assert len(fake_db.saved) == 1
```

pytest

The Testing Pyramid

Most of your confidence should come from many fast, cheap tests

E2E / UI Tests

Few — slow, brittle, expensive

Integration Tests

Some — check modules work together

Unit Tests

Many — fast, isolated, run on every save

Rule of thumb

- Unit tests: run in milliseconds, no I/O
- Integration: real DB/queue, fewer of them
- E2E: full system, reserve for critical paths

Tools That Enforce These Habits

Automate what shouldn't depend on a reviewer remembering to check

Formatter

`black / prettier`

Removes style debates — code always looks the same

Linters

`flake8 / pylint / eslint`

Catches unused variables, complexity, common bugs

Type checker

`mypy / TypeScript`

Catches whole classes of bugs before a test even runs

Coverage tool

`pytest-cov`

Shows which lines have no test touching them at all

Before You Open a Pull Request

A quick self-review checklist — run through this every time

- ✓ Do names explain intent without needing a comment?
- ✓ Does every function do exactly one thing?
- ✓ Is there any copy-pasted logic that should be extracted?
- ✓ Are dependencies (DB, network, clock) passed in, not hard-coded?
- ✓ Can I unit test the core logic without touching a real database?
- ✓ Did I add or update tests for this change?

HANDS-ON

Your Turn: The Refactoring Exercise

You'll receive a small but genuinely messy Python module (order processing) and refactor it end-to-end

1

Rename

Fix unclear variable and function names throughout

2

Split

Break the one giant function into single-purpose functions

3

Modularize

Separate business logic from I/O (printing, "DB" writes)

4

Inject dependencies

Make the data store and notifier swappable, not hard-coded

5

Test

Write unit tests that cover the core logic without real I/O

Full instructions, starter code, and the grading rubric are in the accompanying exercise handout.

Now Go Refactor Something

Clean code is a discipline, not a talent — it gets easier every time you practice it deliberately.

Abinash Kumar Mishra · Co-Founder & CTO, AceRanker AI · @HustlerCoder